

# Can your app charge twice?

A checkpoint is a bookmark. It is not a payment receipt.

---

Predict first: a worker charges a card, crashes, then restarts. What happens next?

# Two records. Two responsibilities.

Workflow record: PAY-17 is pending.

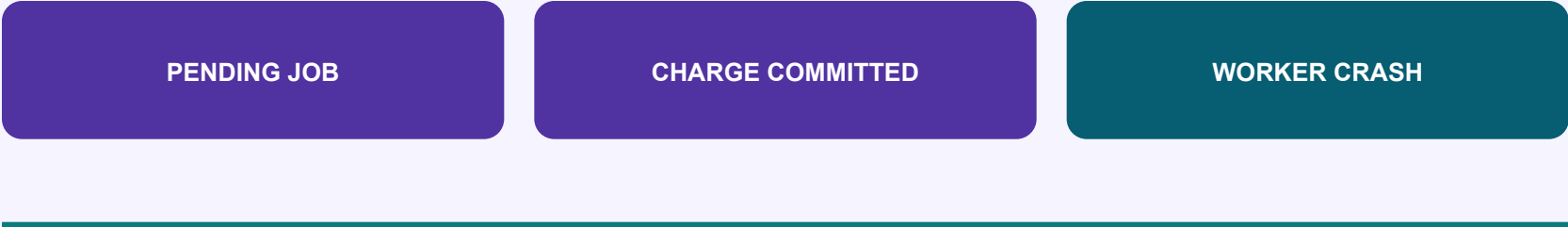
Receiver record: PAY-17 has been charged.

---

A database commit in one service does not automatically update the other service.

# Walk the failure

1. Save pending job PAY-17.
2. Receiver commits the charge.
3. Worker crashes before saving completion.



PENDING JOB

CHARGE COMMITTED

WORKER CRASH

On restart, the worker still sees pending. It cannot infer the receiver did nothing.

# Predict the retry

The worker sends PAY-17 again.  
Will there be one charge or two?

---

Pause here. State what the receiver must remember, and where it must remember it.

# The answer lives at the receiver

With atomic deduplication: return the original receipt.  
Without it: another charge may occur.

SAME OPERATION KEY

DEDUPLICATE: 1 CHARGE

NO DEDUP: 2 POSSIBLE

---

Store the operation key and effect within the same receiver-side atomic boundary.

# A key alone is not enough

Concurrent attempts must not both win.  
The same key with a changed amount must conflict.

---

Enforce uniqueness and compare request identity or payload. A check followed by an unrelated insert can race.

# Test the dangerous gap

Kill the worker after the charge commits, before completion is saved.

Restart it and count durable effects.

---

Expected: one intended payment, one charge, and the same receipt returned on retry.

# The interview answer

Checkpoints preserve progress.

Idempotency controls duplicate effects.

Retries need stable operation identity.

---

Name the guarantee boundary. Do not claim universal exactly-once execution across independent systems.



# Take the idea further

Next: durable agents, transactional outbox, Kafka offsets, and recovery testing.

---

Companion to notebooks 7, 18 and 22. Read the full library for assumptions, code, alternatives and exercises.